1η Άσκηση Στα Λειτουργικά Συστήματα(Περσινή)

Ονοματεπώνυμο: Ιωάννης Δρίβας
ΑΜ : 5216

Κατανόηση του αρχικού κώδικα

Ο αρχικός κώδικας ξεκινά από την main που βρίσκεται στο αρχείο bench.c και έχει τις λειτουργίες write και read που δίνονται σαν όρισμα στη γραμμή εντολών κατά την εκτέλεση του προγράμματος. Όταν δίνουμε την επιλογή write το πρόγραμμα καλεί με τη σειρά την _write_test που βρίσκεται στην kiwi.c, ξεκινά τη χρονομέτρηση και κάνοντας μια επανάληψη όσες φορές είναι ο αριθμός των στοιχείων του δώσαμε (count) καλείται η db_add η οποία είναι υπεύθυνη να γράψει τα στοιχεία στο memtable. Το memtable αποθηκεύει τα στοιχεία προσωρινά στη ram για γρήγορη προσπέλαση και όταν γεμίσει πραγματοποιείται ένα compaction και μεταφέρονται τα στοιχεία σε ένα sst file στον σκληρό δίσκο. Έπειτα και τα sst αρχεία όταν φτάσουν ένα συγκεκριμένο όριο δεδομένων πραγματοποιούν compaction και κατεβαίνουν σε επόμενο level έως ότου εξαντληθεί η διαθέσιμη ιεραρχία επιπέδων (levels) του δέντρου. Κατά τη διάρκεια χρήσης του memtable υπάρχει και ένα log αρχείο το οποίο πραγματοποιεί backup του memtable στο σκληρό δίσκο για τυχόν σφάλματα κατά τον compaction. Αντίστοιχα οι _read_test καλεί την db_get η οποία είναι υπεύθυνη να διαβάσει στοιχεία αρχικά από το μενού του memtable και εν συνεχεία από το sst αρχεία.

Db.h

Στο συγκεκριμένο header file προσθέτουμε τη βιβλιοθήκη διαχείρισης νημάτων pthreads και ορίζουμε μέσα στο struct του db τις μεταβλητές των ενεργών γραφέων και αναγνωστών, τις μεταβλητές των εγγραφέων και αναγνωστών που περιμένουν καθώς και mutex_lock για την ασφάλιση της κρίσιμης περιοχής και condition_locks για το εάν

επιτρέπεται η εγγραφή ή η ανάγνωση. Παρακάτω είναι και ο αντίστοιχος κώδικας:

```
#include <pthread.h> // added the pthread library to initialize our mutex values

typedef struct _db {
//    char basedir[MAX_FILENAME];
    char basedir[MAX_FILENAME+1];
    SST* sst;
    MemTable* memtable;

    int active_readers; // count the active readers
    int active_writers; // count the active writers
    int waiting_readers; // count the waiting readers
    int waiting_writers; // count the waiting writers
    pthread_mutex_t lock; // mutex to protect the counters
    pthread_cond_t safeToRead; // condition signal when its safe to read
    pthread_cond_t safeToWrite; // condition signal when its safe to write
} DB;
```

Db.c

Στο συγκερκριμένο file θα υλοποιήσουμε τα νήματά μας με προτεραιότητα στους εγγραφείς και είσοδο πολλαπλών αναγνωστών αλλά μόνο ενός εγγραφέα. Η προτεραιότητα στους εγγραφείς μας παρέχει την αποφυγή του starvation καθώς τις περισσότερες φορές αναζητούμε εγγραφές και δεν προσθέτουμε, δηλαδή θα εκτελούνταν μόνιμα η db_get και η db_add, θα έμενε πάντα για το τέλος με αποτέλεσμα από ένα σημείο και μετά να διαβάζουμε μόνιμα παλιά δεδομένα. Η είσοδος πολλών αναγνωστών μας αυξάνει τη ταχύτητα του προγράμματός μας καθώς επειδή δεν επέρχεται αλλαγή στις εγγραφές μπορούμε να έχουμε πολλές ταυτόχρονες αναγνώσεις. Στην εγγραφή, θέλουμε μόνο ένα ενεργό εγγραφέα και κανέναν αναγνώστη μέσα καθώς πολλαπλοί εγγραφείς μπορούν να γράψουν πάνω στην ίδια εγγραφή χωρίς να προλάβουμε να δούμε τι είχε γράψει ο πρώτος ενώ η χρήση αναγνωστών καθώς υπάρχει ενεργός εγγραφέας μπορεί να διαβάσει δεδομένα που δεν έχει προλάβει να γράψει ο εγγραφέας.

Στην συνάρτηση db_open_ex αρχικοποιούμε τις μεταβλητές που ορίσαμε στο db.h για να τις χρησιμοποιήσουμε στο db_add και db_get όπως φαίνεται παρακάτω:

```c
DB* db_open_ex(const char* basedir, uint64_t cache_size)
{
    DB* self = calloc(1, sizeof(DB));

    if (!self)
        PANIC("NULL allocation");

    self->active_readers = 0; // set active readers to 0
    self->active_writers = 0; // set active writers to 0
    self->waiting_readers = 0; // set waiting readers to 0
    self->waiting_writers = 0; // set waiting writers to 0
    pthread_mutex_init(&self->lock, NULL); // initialize the mutex lock
    pthread_cond_init(&self->safeToRead, NULL); //initialize the condition safeToRead
    pthread_cond_init(&self->safeToWrite, NULL); //initialize the condition safeToWrite

    strncpy(self->basedir, basedir, MAX_FILENAME);
    self->sst = sst_new(basedir, cache_size);

    Log* log = log_new(self->sst->basedir);
    self->memtable = memtable_new(log);

    return self;
}
```

Στη συνάρτηση db_close εκτελούμε mutex_destroy και condition_destroy για να ελευθερώσουμε τις μεταβλητές που αρχικοποιήσαμε. Αυτό γίνεται διότι η ήδη υπάρχουσα εντολή free(self) ελευθερώνει το χώρο που δεσμεύτηκε στη ram αλλά όχι τους πόρους που δέσμευσαν τα mutex και condition από το λειτουργικό σύστημα.

```c
void db_close(DB *self)
{
    INFO("Closing database %d", self->memtable->add_count);

    if (self->memtable->list->count > 0)
    {
        sst_merge(self->sst, self->memtable);
        skiplist_release(self->memtable->list);
        self->memtable->list = NULL;
    }

    pthread_mutex_destroy(&self->lock); // destroy the mutex lock
    pthread_cond_destroy(&self->safeToRead); // destroy the condition safeToRead
    pthread_cond_destroy(&self->safeToWrite); // destroy the condition safeToWrite

    sst_free(self->sst);
    log_remove(self->memtable->log, self->memtable->lsn);
    log_free(self->memtable->log);
    memtable_free(self->memtable);
    free(self);
}
```

Για την υλοποίηση της db_add αρχικά παίρνουμε ένα lock για να κλειδώσουμε τη κρίσιμη περιοχή. Στη περιοχή αυτή ελέγχουμε εάν υπάρχει ενεργός εγγραφέας ή αναγνώστης

και εάν ναι, αυξάνουμε τον αριθμό των εγγραφέων που περιμένουμε και κάνουμε condition_wait για τη μεταβλητή safeToWrite καθώς δεν μπορούμε να ακόμη να γράψουμε. Έπειτα μόλις γίνει signal μειώνουμε κατά μια μονάδα τους γραφείς που περιμένουν. Μόλις δεν βρίσκεται κανένας στην βάση αυξάνουμε κατά μια μονάδα τους ενεργούς γραφείς και βγαίνουμε από τη κρίσιμη περιοχή κάνοντας mutex_unlock.

```
int db_add(DB* self, Variant* key, Variant* value)
{
    pthread_mutex_lock(&self->lock); // acquire the lock for the critical section
    while (self->active_writers > 0 || self->active_readers > 0)
    {
        self->waiting_writers++; // increment waiting writers
        pthread_cond_wait(&self->safeToWrite, &self->lock); // wait until it's safe to write
        self->waiting_writers--; // decrement waiting writers after being signaled
    }

    self->active_writers++; // increment active writers
    pthread_mutex_unlock(&self->lock); // release the lock from exiting the critical section
```

Έπειτα εκτελούμε τον ήδη υπάρχοντα κώδικα της db_add και προσθέτουμε μια μεταβλητή added που ισούται με το memtable_add. Η προηγούμενη συνάρτηση πριν βρισκόταν στο return της db_add και πλέον έχει μεταφερθεί πάνω μιας και θα ξαναμπούμε σε κρίσιμη περιοχή.

```
if (memtable_needs_compaction(self->memtable))
{
    INFO("Starting compaction of the memtable after %d insertions and %d deletions",
         self->memtable->add_count, self->memtable->del_count);
    sst_merge(self->sst, self->memtable);
    memtable_reset(self->memtable);
}

int added = memtable_add(self->memtable, key, value); // add the key-value pair to the memtable
```

Τέλος ξανά κλειδώνουμε τη κρίσιμη περιοχή με mutex_lock και μειώνουμε τους ενεργούς γραφείς μιας και γράψαμε την εγγραφή μας. Μετά ελέγχουμε εάν περιμένουν άλλοι εγγραφείς και εάν ναι κάνουμε signal να εισέλθει ο επόμενος. Εάν δεν υπάρχει άλλος εγγραφέας ελέγχουμε εάν υπάρχει ενεργός αναγνώστης και εάν ναι κάνουμε **broadcast** για να εισέλθουν όλοι οι αναγνώστες. Τέλος ξεκλειδώνουμε τη κρίσιμη περιοχή και επιστρέφουμε την μεταβλητή added.

```
pthread_mutex_lock(&self->lock); // acquire the lock for the critical section
self->active_writers--; // decrement active writers after finishing the write operation
if (self->waiting_writers > 0) // if there are waiting writers, signal one writer to proceed
{
    pthread_cond_signal(&self->safeToWrite); // signal if any other writers are waiting
}
else if (self->waiting_readers > 0) // if there are waiting readers and no waiting writers, signal all readers
{
    pthread_cond_broadcast(&self->safeToRead); // broadcast all waiting readers if there are any
}
pthread_mutex_unlock(&self->lock); // release the lock from exiting the critical section

return added; // return the result of the add operation
```

Για την db_get πάλι παίρνουμε mutex_lock και στη while ελέγχουμε εάν υπάρχει ενεργός ή αναμένων γραφέας και αν ναι, αυξάνουμε τους αναγνώστες που περιμένουν δίνοντας έτσι προτεραιότητα στους γραφείς. Μόλις βγούμε από την while είναι σειρά εκτέλεσης για αναγνώστη και αυξάνουμε τους ενεργούς γραφείς και βγαίνουμε από την κρίσιμη περιοχή.

```
int db_get(DB* self, Variant* key, Variant* value)
{
    pthread_mutex_lock(&self->lock); // acquire the lock for the critical section
    while (self->active_writers > 0 || self->waiting_writers > 0)
    {
        self->waiting_readers++; // increment waiting readers
        pthread_cond_wait(&self->safeToRead, &self->lock); // wait until it's safe to read
        self->waiting_readers--; // decrement waiting readers after being signaled
    }
    self->active_readers++; // increment active readers
    pthread_mutex_unlock(&self->lock); // release the lock from exiting the critical section
```

Όπως προσθέσαμε την added στην db_add έτσι προσθέτουμε και εδώ τη μεταβλητή got και εκτελούμε τον ήδη υπάρχοντα κώδικα της db_get. Μια διαφορά με πριν είναι ότι εδώ ελέγχουμε πρώτα εάν η τιμή είναι στο memtable αλλιώς ψάχνουμε στο sst επομένως τροποποιήσαμε τον κώδικα για να παίρνει η μεταβλητή got τη κατάλληλη τιμή κάθε φορά.

```
pthread_mutex_unlock(&self->lock); // release the lock from exiting the critical section
int got = 0;

if (memtable_get(self->memtable->list, key, value) == 1)
{
        got = 1; // if the key is found in the memtable, set got to 1
}else{
    got = sst_get(self->sst, key, value); // if the key is not found in the memtable, try to get it from the SST files
}
```

Τέλος ξανά κλειδώνουμε τη περιοχή για να ελαττώσουμε κατά έναν τους αναγνώστες αφού έψαξε για την εγγραφή που ήθελε. Μετά ελέγχουμε εάν υπάρχει αναμένων γραφέας και να μην υπάρχει ενεργός αναγνώστης, δίνοντας έτσι προτεραιότητα στους γραφείς να μπουν και να γράψουν κάνοντας signal έναν γραφέα. Τέλος ξεκλειδώνουμε τη περιοχή και επιστρέφουμε τη τιμή του got.

```
pthread_mutex_lock(&self->lock); // acquire the lock for the critical section
self->active_readers--; // decrement active readers after finishing the read operation
if (self->waiting_writers > 0 && self->active_readers == 0) // if there are waiting writers and no active readers, signal one writer
{
    pthread_cond_signal(&self->safeToWrite); // signal if any writers are waiting
}
pthread_mutex_unlock(&self->lock); // release the lock from exiting the critical section
return got; // return the result of the get operation
```

Bench.h

Στο bench.h προσθέτουμε τις παρακάτω γραμμές:

```
void _write_test(long int count, int r, int num_threads); // Added num_threads parameter
void _read_test(long int count, int r, int num_threads); // Added num_threads parameter
```

Το παραπάνω μας επιτρέπει να έχουμε πρόσβαση όταν καλούμε την _write_test και _read_test στον αριθμό των νημάτων που θα δίνουμε από το τερματικό.

Kiwi.c

Αρχικά προσθέτουμε τη βιβλιοθήκη pthreads και stdlib για να έχουμε πρόσβαση στις μεθόδους τους. Έπειτα δημιουργούμε ένα struct όπου θα περνάμε τις απαραίτητες μεταβλητές για κάθε νήμα οι οποίες είναι:

• Thread_id: αναγνωριστικό id του νήματος
• Start_index: η θέση του πρώτου κλειδιού που θα ξεκινήσει το νήμα
• Count: ο αριθμός των κλειδιών που θα προσπελάσει το νήμα
• R: Σημαία για τυχαίο κλειδί ή όχι
• Found: αριθμός κλειδιών που βρέθηκαν κατά την ανάγνωση για ένα νήμα

```
#include <pthread.h> // for thread methods
#include <stdlib.h> // for malloc and free
```

```
typedef struct {
    int thread_id;       // Unique identifier for the thread
    long int start_index; // starting key index
    long int count;       // number of keys to process
    int r;                // random key flag
    DB* db;               // pointer to the database
    int found;            // number of keys found
    int read_percent;       // percentage of read operations for mixed workload
} ThreadArg;
```

Το επόμενο βήμα είναι να γράψουμε τις _write και _read που αναφέρονται για ένα νήμα οι οποίες έχουν παρόμοια δομή. Αρχικά περνάμε σαν ορίσματα το struct που δημιουργήσαμε και αναθέτουμε τις τιμές του struct σε τοπικές μεταβλητές της συνάρτησης για να είναι ευανάγνωστες. Από εδώ και πέρα το μεγαλύτερο ποσοστό του κώδικα είναι μεταφορά από τις _write_test και _read_test στις δικές μας μεθόδους. Στην

_write δημιουργούμε τις μεταβλητές key,val,sk,sv ενώ στην _read παραλείπουμε το val καθώς δεν χρειάζεται στην ανάγνωση μιας και υπάρχει ήδη τιμή και δεν ορίζουμε νέα. Έπειτα και οι δύο μέθοδοι έχουν μια for loop όπου κάθε νήμα ξεκινά από το δικό του start_index και φτάνει έως το start_index+count εκτελώντας είτε το db_add είτε το db_get για τον αριθμό εγγραφών που του έχει ανατεθεί. Μέσα μηδενίζουμε τους buffers για τα key,val,sk,sv στο _write και μόνο το key για το val. Ελέγχουμε σε τι mode είμαστε για τη παραγωγή τυχαίου κλειδιού, ανανεώνουμε τις τιμες sk,sv (για το _read μόνο sk) και εκτελούμε στην _write το db_add ενώ στην _read το db_get. Στην τελευταία ελέγχουμε εάν βρήκαμε εγγραφή για το συγκεκριμένο κλειδί και αν ναι, αυξάνουμε το found του νήματος και καθαρίζουμε το χώρο που δέσμευσε στη μνήμη.

```
void* _write(void* arg) {
    ThreadArg* thread_arg = (ThreadArg*)arg; // Cast the argument to ThreadArg structure
    long int start_index = thread_arg->start_index; // Get the starting index for this thread
    long int count = thread_arg->count; // Get the number of keys this thread should process
    int r = thread_arg->r;  // Get the random key flag for this thread
    DB* db = thread_arg->db;    // Get the database pointer for this thread

    char key[KSIZE + 1];    //moved from _write_test since each thread needs its own key buffer
    char val[VSIZE + 1];    //moved from _write_test since each thread needs its own value buffer
    Variant sk, sv;         //moved from _write_test since each thread needs its own Variant structures

    for (long int i = start_index; i < start_index + count; i++) { // Loop through the assigned key range for this thread
        memset(key, 0, KSIZE + 1);  // Clear the key buffer for the current thread
        memset(val, 0, VSIZE + 1); // Clear the value buffer for the current thread
        memset(&sk, 0, sizeof(Variant)); // Clear the key Variant structure for the current thread
        memset(&sv, 0, sizeof(Variant)); // Clear the value Variant structure for the current thread

        if (r)  // If random key flag is set, generate a random key
            _random_key(key, KSIZE);
        else
            snprintf(key, KSIZE, "key-%ld", i);
        snprintf(val, VSIZE, "val-%ld", i);
        sk.length = KSIZE;  // Set the length of the key Variant structure for the current thread
        sk.mem = key;       // Set the memory pointer of the key Variant structure for the current thread
        sv.length = VSIZE;  // Set the length of the value Variant structure for the current thread
        sv.mem = val;       // Set the memory pointer of the value Variant structure for the current thread

        db_add(db, &sk, &sv);   // Add the key-value pair to the database using the current thread's key and value
    }
    return NULL; // Return NULL to indicate the thread has finished its work
}
```

```c
void* _read(void* arg) {
    ThreadArg* thread_arg = (ThreadArg*)arg;    // Cast the argument to ThreadArg structure
    long int start_index = thread_arg->start_index; // Get the starting index for this thread
    long int count = thread_arg->count; // Get the number of keys this thread should process
    int r = thread_arg->r;      // Get the random key flag for this thread
    thread_arg->found = 0; //Get the initial found count for this thread
    int ret;    // Variable to store the return value of db_get
    DB* db = thread_arg->db; // Get the database pointer for this thread

    char key[KSIZE + 1];    //moved from _read_test since each thread needs its own key buffer
    Variant sk, sv;     //moved from _read_test since each thread needs its own Variant structures

    for (long int i = start_index; i < start_index + count; i++) {  // Loop through the assigned key range for this thread
        memset(key, 0, KSIZE + 1);  // Clear the key buffer for the current thread
        memset(&sk, 0, sizeof(Variant)); // Clear the key Variant structure for the current thread
        memset(&sv, 0, sizeof(Variant)); // Clear the key and value Variant structures for the current thread

        if (r)  // If random key flag is set, generate a random key
            _random_key(key, KSIZE);
        else
            snprintf(key, KSIZE, "key-%ld", i);

        sk.length = KSIZE;  // Set the length of the key Variant structure for the current thread
        sk.mem = key;   // Set the memory pointer of the key Variant structure for the current thread
        //read doesnt have sv since we are only interested in the key and it will bring the value if the key is found
        ret = db_get(db, &sk, &sv); // Attempt to get the value associated with the key from the database using the current thread's key
        if (ret) {  // If the key is found in the database, increment the found count for this thread
            thread_arg->found++;
        }
        if (sv.mem) { // Free the memory allocated for the value if it was found
                free(sv.mem);
        }
    }
    return NULL; // Return NULL to indicate the thread has finished its work
}
```

Στην _write_test και στην _read_test έχουμε ίδιο κώδικα πέρα από τη μεταβλητή total_found της δεύτερης που αποθηκεύει τον αριθμό των συνολικών κλειδιών που βρέθηκαν στη βάση. Προσθέτουμε στα ορίσματα τον αριθμό των threads που θέλουμε να χρησιμοποιήσουμε. Κρατήσαμε μεταβλητές που ήδη υπήρχαν και όσες μεταφέραμε στις _write,_read τις αφαιρέσαμε. Μετά δεσμεύουμε χώρο για τα νήματα και τα stuct των νημάτων ενώ θέτουμε τη μεταβλητή count_per_thread και δείχνει τον αριθμό των εγγραφών που θα τρέξει το κάθε thread. Έπειτα σε ένα for loop που θα τρέξει όσες φορές είναι τα νήματα που έχουμε αρχικοποιούμε τις τιμές του struct. Για το start_index το θέτουμε ίσο με την επανάληψη που βρισκόμαστε επί το count_per_thread ενώ για το count ελέγχουμε εάν είμαστε στο τελευταίο νήμα και του δίνουμε όσα περίσσεψαν καθώς ο συνολικός αριθμός των εγγραφών μπορεί να μην διαιρείται ακριβώς με τα νήματα που έχουμε. Έπειτα τα δημιουργούμε με την pthread_create και σαν όρισμα παίρνει το νήμα, τα ορίσματά του και την αντίστοιχη μέθοδο που θα εκτελέσει (_write ή _read). Μετά τα κάνουμε pthread_join όπου περιμένουμε μέχρι να τελειώσουν, προσθέτουμε στην total_found τα found του κάθε νήματος και στο τέλος ελευθερώνουμε το χώρο που δεσμεύσαμε.

```c
void _write_test(long int count, int r, int num_threads) // Added num_threads parameter
{
    double cost;
    long long start,end;
    DB* db;

    db = db_open(DATAS);

    start = get_ustime_sec();

    pthread_t* threads = malloc(num_threads * sizeof(pthread_t));   // Allocate memory for thread identifiers
    ThreadArg* args = malloc(num_threads * sizeof(ThreadArg));      // Allocate memory for thread arguments
    long int count_per_thread = count / num_threads;

    for (int i = 0; i < num_threads; i++) { // Create threads for writing
        args[i].start_index = i * count_per_thread;
        if (i == num_threads - 1) {
            args[i].thread_id = i; // Set the thread ID for this thread
            args[i].count = count - (count_per_thread * i); // Handle remaining keys for the last thread
        } else {
            args[i].count = count_per_thread;
        }
        args[i].r = r;
        args[i].db = db;
        pthread_create(&threads[i], NULL, _write, &args[i]);
    }

    for (int i = 0; i < num_threads; i++) { // Wait for all threads to finish
        pthread_join(threads[i], NULL);
    }

    free(threads);  // Free the allocated memory for thread identifiers
    free(args);     // Free the allocated memory for thread arguments

    db_close(db);

    end = get_ustime_sec();
    cost = end -start;

    printf(LINE);
    printf("|Random-Write   (done:%ld): %.6f sec/op; %.1f writes/sec(estimated); cost:%.3f(sec);\n"
        ,count, (double)(cost / count)
        ,(double)(count / cost)
        ,cost);
}
```

```c
void _read_test(long int count, int r, int num_threads) // Added num_threads parameter
{
    int total_found = 0; // Initialize total found count
    double cost;
    long long start,end;
    DB* db;

    db = db_open(DATAS);
    start = get_ustime_sec();
    pthread_t* threads = malloc(num_threads * sizeof(pthread_t)); // Allocate memory for thread identifiers
    ThreadArg* args = malloc(num_threads * sizeof(ThreadArg));    // Allocate memory for thread arguments
    long int count_per_thread = count / num_threads; // Calculate the number of keys each thread should process

    for (int i = 0; i < num_threads; i++) { // Create threads for reading
        args[i].thread_id = i; // Set the thread ID for this thread
        args[i].start_index = i * count_per_thread; // Set the starting index for each thread
        if (i == num_threads - 1) {
            args[i].count = count - (count_per_thread * i); // Handle remaining keys for the last thread
        } else {
            args[i].count = count_per_thread;
        }
        args[i].r = r;
        args[i].db = db;
        pthread_create(&threads[i], NULL, _read, &args[i]);
    }

    for (int i = 0; i < num_threads; i++) { // Wait for all threads to finish
        pthread_join(threads[i], NULL);
        total_found += args[i].found; // Accumulate the total found count from all threads
    }

    free(threads); // Free the allocated memory for thread identifiers
    free(args);    // Free the allocated memory for thread arguments

    db_close(db);

    end = get_ustime_sec();
    cost = end - start;
    printf(LINE);
    printf("|Random-Read    (done:%ld, found:%d): %.6f sec/op; %.1f reads /sec(estimated); cost:%.3f(sec)\n",
        count, total_found,
        (double)(cost / count),
        (double)(count / cost),
        cost);
}
```

Στην read_write για τον συνδυασμό αναγνωστών και γραφέων έχουμε ολόιδιο κώδικα με την _read έχοντας προσθέσει τα εξής:

```c
unsigned int seed = time(NULL) ^ thread_arg->thread_id; // Initialize a unique seed for random number generation for this thread
```

Όπου δημιουργούμε μια μεταβλητή για την τυχαία παραγωγή νημάτων.

```c
if (rand_r(&seed) % 100 < thread_arg->read_percent) { // Perform a read operation based on the specified read percentage
    ret = db_get(db, &sk, &sv); // Attempt to get the value associated with the key from the database using the current thread's key
    if (ret) { // If the key is found in the database, increment the found count for this thread
        found++;
    }
} else {
    snprintf(val, VSIZE, "val-%ld", i);
    sv.length = VSIZE;
    sv.mem = val;
    db_add(db, &sk, &sv); // Add the key-value pair to the database using the current thread's key and value
}
```

Ελέγχουμε το ποσοστό και αντίστοιχα κάνουμε db_get ή db_add, όπου στην db_add θέτουμε και τιμές στο sv.

```
void read_write(void* arg){
    ThreadArg* thread_arg = (ThreadArg*)arg; // Cast the argument to ThreadArg structure
    long int start_index = thread_arg->start_index; // Get the starting index for this thread
    long int count = thread_arg->count; // Get the number of keys this thread should process
    int r = thread_arg->r;  // Get the random key flag for this thread
    DB* db = thread_arg->db;    // Get the database pointer for this thread
    thread_arg->found = 0; //Get the initial found count for this thread
    int ret;

    char key[KSIZE + 1];    //moved from _write_test since each thread needs its own key buffer
    char val[VSIZE + 1];    //moved from _write_test since each thread needs its own value buffer
    Variant sk, sv;         //moved from _write_test since each thread needs its own Variant structures
    unsigned int seed = time(NULL) ^ thread_arg->thread_id; // Initialize a unique seed for random number generation for this thread

    for (long int i = start_index; i < start_index + count; i++) { // Loop through the assigned key range for this thread
        memset(key, 0, KSIZE + 1);  // Clear the key buffer for the current thread
        memset(val, 0, VSIZE + 1); // Clear the value buffer for the current thread
        memset(&sk, 0, sizeof(Variant)); // Clear the key Variant structure for the current thread
        memset(&sv, 0, sizeof(Variant)); // Clear the value Variant structure for the current thread

        if (r)  // If random key flag is set, generate a random key
            _random_key(key, KSIZE);
        else
            snprintf(key, KSIZE, "key-%ld", i);


        sk.length = KSIZE;  // Set the length of the key Variant structure for the current thread
        sk.mem = key;       // Set the memory pointer of the key Variant structure for the current thread


        if (rand_r(&seed) % 100 < thread_arg->read_percent) { // Perform a read operation based on the specified read percentage
            ret = db_get(db, &sk, &sv); // Attempt to get the value associated with the key from the database using the current thread's key
            if (ret) { // If the key is found in the database, increment the found count for this thread
                thread_arg->found++;
            }
            if (sv.mem) { // Free the memory allocated for the value if it was found
                free(sv.mem);
            }
        } else {
            snprintf(val, VSIZE, "val-%ld", i);
            sv.length = VSIZE;
            sv.mem = val;
            db_add(db, &sk, &sv); // Add the key-value pair to the database using the current thread's key and value
        }
    }
    return NULL; // Return NULL to indicate the thread has finished its work
}
```

Η read_write έχει ίδια λογική με την _read_test απλά έχουμε προσθέσει σαν όρισμα και το read_percent:

```c
void _readwrite_test(long int count, int r, int num_threads, int read_percent) // Added num_threads and read_percent parameters
{
    int total_found = 0;    // Initialize total found count for read operations
    double cost;
    long long start, end;
    DB* db;

    db = db_open(DATAS);
    start = get_ustime_sec();

    pthread_t* threads = malloc(num_threads * sizeof(pthread_t)); // Allocate memory for thread identifiers
    ThreadArg* args = malloc(num_threads * sizeof(ThreadArg)); // Allocate memory for thread arguments

    long int count_per_thread = count / num_threads; // Calculate the number of keys each thread should process

    for (int i = 0; i < num_threads; i++) { // Create threads for mixed read/write operations
        args[i].thread_id = i;
        args[i].start_index = i * count_per_thread;
        if (i == num_threads - 1) {
            args[i].count = count - (count_per_thread * i); // Handle remaining keys for the last thread
        } else {
            args[i].count = count_per_thread;
        }
        args[i].r = r;
        args[i].db = db;
        args[i].read_percent = read_percent;

        pthread_create(&threads[i], NULL, read_write, &args[i]);
    }

    for (int i = 0; i < num_threads; i++) { // Wait for all threads to finish and accumulate the total found count
        pthread_join(threads[i], NULL);
        total_found += args[i].found;
    }

    free(threads); // Free the allocated memory for thread identifiers
    free(args); // Free the allocated memory for thread arguments
    db_close(db);

    end = get_ustime_sec();
    cost = end - start;

    printf(LINE);
    printf("|ReadWrite %d%% R \t(ops:%ld found:%d): %.6f sec/op; %.1f ops/sec; cost:%.3f(sec);\n"
        ,read_percent, count, total_found
        ,(double)(cost / count)
        ,(double)(count / cost)
        ,cost);
}
```

Sst.c

Στο συγκεκριμένο αρχείο παρατηρήσαμε πως υπήρχε πρόβλημα Deadlock μεταξύ των locks sst->lock και sst->cv_lock. Στη συγκεκριμένη υλοποίηση εάν ένα νήμα A καλέσει την merge_thread παίρνει ένα cv_lock και στο condition_timewait κοιμάται αφήνοντας το lock. Ένα νήμα B εκτελεί sst_get και παίρνει sst_lock. Το νήμα A ξυπνά ξαναπαίρνει το cv_lock και εκτελεί την sst_compact που αυτή εκτελεί την compact_install και χρειάζεται το sst->lock άρα περιμένει. Το νήμα B εκτελεί _schedule_compaction και χρειάζεται το sst->cv_lock που οδηγεί σε **αδιέξοδο**. Για να το λύσουμε στην merge_thread αφήνουμε προσωρινά το cv_lock και το ξαναπαίρνουμε για να ξεφύγουμε από το deadlock.

```c
            }

            DEBUG("Compactiong due to many files in level 0");

            pthread_mutex_unlock(&sst->cv_lock); // Unlock before evaluating compaction and performing compaction to allow other threads to proceed with their work while we are doing the compaction
            _evaluate_compaction(sst);
            sst_compact(sst);

            pthread_mutex_lock(&sst->cv_lock); // Lock again after compaction is done to check the merge state and continue with the next iteration of the loop
        }
```

```
            pthread_mutex_unlock(&sst->cv_lock);// Unlock before merging to allow other threads to proceed with their work while we are doing the merge
            sst_merge_real(sst, sst->immutable_list);
            pthread_mutex_lock(&sst->cv_lock); // Lock again after merge is done to check the merge state and continue with the next iteration of the loop

            INFO("Merge successfully completed. Releasing the skiplist");
            skiplist_release(sst->immutable_list);
```

```
        pthread_mutex_unlock(&sst->cv_lock); // Unlock before evaluating compaction and performing compaction to allow other threads to proceed with their work while we are doing the compaction
        sst_compact(sst);
        pthread_mutex_lock(&sst->cv_lock); // Lock again after compaction is done to check the merge state and continue with the next iteration of the loop
        compacted = 1;
    }
```

Bench.c

Στο συγκεκριμένο αρχείο στη main προσθέσαμε κατάλληλες αλλαγές για να υποστηρίζεται το readwrite και η προσθήκη επιπλέον μεταβλητών στη γραμμή εντολών. Συγκεκριμένα προσθέσαμε τη μεταβλητή number_of_threads και μετατοπίσαμε το r έξω από τις if. Για να πάρουμε τα δεδομένα από τη γραμμή εντολών χρησιμοποιούμε την atoi και τα περνάμε στις αντίστοιχες μεταβλητές. Τέλος ελέγχουμε σε ποια επιλογή είμαστε και την εκτελούμε.

```
int main(int argc,char** argv)
{
    long int count;

    int number_of_threads = 1; // Default to 1 thread


    srand(time(NULL));
    if (argc < 6) {
        fprintf(stderr,"Usage: db-bench <write | read | readwrite> <count> <number_of_threads> <read_percent(WRITE null IF NOT USING readwrite)> <random(WRITE 1 FOR RANDOM KEYS, 0 FOR SEQUENTIAL)>\n");
        exit(1);
    }

    count = atoi(argv[2]); //number of keys to process
    number_of_threads = atoi(argv[3]); // number of threads for write test
    int r = atoi(argv[5]);  // random key flag
    _print_environment();
    _print_header(count);

    if (strcmp(argv[1], "write") == 0) {
        if (strcmp(argv[4], "null") != 0) { //read percentage, not used in write test
            fprintf(stderr,"Usage for write test: db-bench write <count> <number_of_threads> null <random>\n");
            exit(1);
        }
        _write_test(count, r, number_of_threads); // Pass number of threads to the write test
    } else if (strcmp(argv[1], "read") == 0) {
        if (strcmp(argv[4], "null") != 0) { //read percentage, not used in read test
            fprintf(stderr,"Usage for read test: db-bench read <count> <number_of_threads> null <random>\n");
            exit(1);
        }
        _read_test(count, r, number_of_threads); // Pass number of threads to the read test
    } else if (strcmp(argv[1], "readwrite") == 0) {
        int read_percent = atoi(argv[4]); // percentage of read operations for mixed workload
        if (read_percent < 0 || read_percent > 100) { // Validate read percentage input for read/write test
            fprintf(stderr,"Read percentage must be between 0 and 100 for readwrite test\n");
            exit(1);
        }
        _readwrite_test(count, r, number_of_threads, read_percent); // Pass number of threads and read percentage to the read/write test
    } else {
        fprintf(stderr,"Usage: db-bench <write | read | readwrite> <count> <number_of_threads> <read_percent(WRITE null IF NOT USING readwrite)> <random(WRITE 1 FOR RANDOM KEYS, 0 FOR SEQUENTIAL)>\n");
        exit(1);
    }

    return 1;
}
```

Benchmarks

Read-Writes

Total Ops	R/W(%)	Threads	Throughput	Avg Latency (us)	Read Latency(us)	Write Latency(us)	Time (s)
20K	80/20	3	378201	6.44	5.53	10.11	0.053
20K	80/20	5	318862	11.45	11.00	13.32	0.063

20K	80/20	10	324638	17.64	18.43	14.51	0.062
50K	80/20	3	199075	14.40	12.60	21.67	0.251
50K	80/20	5	300973	12.74	12.70	12.91	0.166
50K	80/20	10	342445	19.52	21.30	12.32	0.146
100K	80/20	3	257493	10.0	9.34	12.64	0.388
100K	80/20	5	171251	27.05	27.19	26.45	0.584
100K	80/20	10	171219	46.72	51.25	28.72	0.584
20K	50/50	3	165405	16.85	22.34	11.38	0.121
20K	50/50	5	221769	19.30	30.94	7.73	0.090
20K	50/50	10	194437	38.70	67.15	10.20	0.103
50K	50/50	3	153427	17.49	24.29	10.69	0.326
50K	50/50	5	139139	29.74	45.03	14.55	0.359
50K	50/50	10	128650	50.31	83.80	16.87	0.389
100K	50/50	3	134608	21.48	29.63	13.40	0.743
100K	50/50	5	118169	38.23	57.96	18.39	0.846
100K	50/50	10	118844	75.65	128.95	22.52	0.841
20K	20/80	3	180054	15.18	49.83	6.54	0.111
20K	20/80	5	142695	31.42	108.14	11.96	0.140
20K	20/80	10	130016	56.52	235.57	13.13	0.154
50K	20/80	3	132984	21.64	68.22	9.91	0.376
50K	20/80	5	151910	28.68	107.85	8.58	0.329
50K	20/80	10	127170	59.93	247.69	13.69	0.393
100K	20/80	3	113212	25.70	84.97	10.89	0.883
100K	20/80	5	110230	41.63	161.51	11.57	0.907
100K	20/80	10	104241	86.65	343.52	22.34	0.959

Reads

Total Ops	Threads	Throughput	Read Latency (us)	Total Time (s)
20K	3	211338	12.67	0.095
20K	5	291817	12.71	0.069
20K	10	307796	19.50	0.065
50K	3	233965	12.30	0.214
50K	5	317499	12.94	0.157
50K	10	414154	17.34	0.121
100K	3	244878	11.87	0.408
100K	5	376391	10.95	0.266
100K	10	433184	15.96	0.231

Writes

Total Ops	Threads	Throughput	Read Latency (us)	Total Time (s)
20K	3	162860	16.97	0.123
20K	5	198874	22.32	0.101
20K	10	129910	52.33	0.154
50K	3	148892	18.60	0.336
50K	5	101262	43.44	0.494
50K	10	87297	87.45	0.573
100K	3	129341	21.94	0.773
100K	5	90735	43.47	1.102
100K	10	96633	86.80	1.035

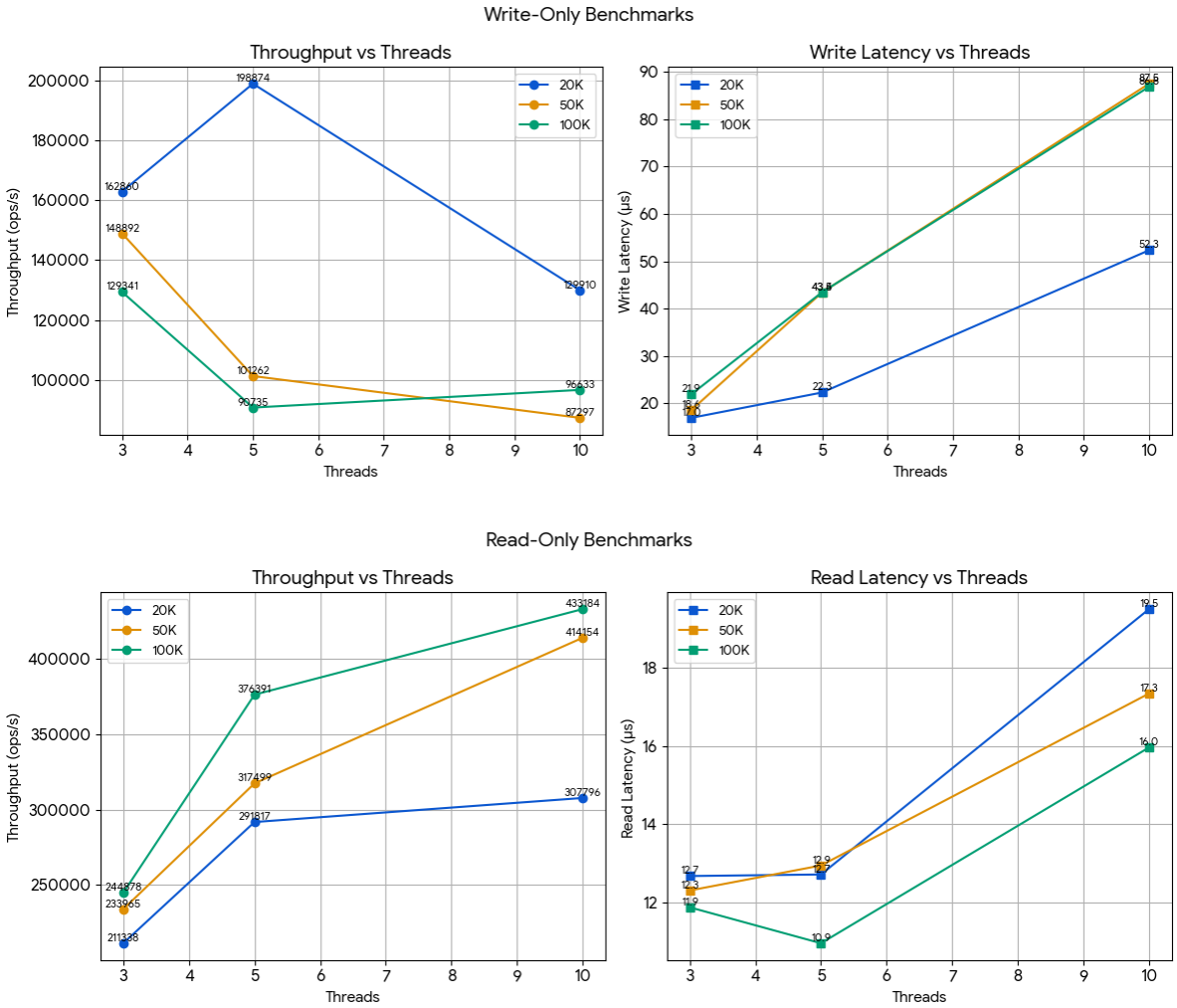
Evaluation

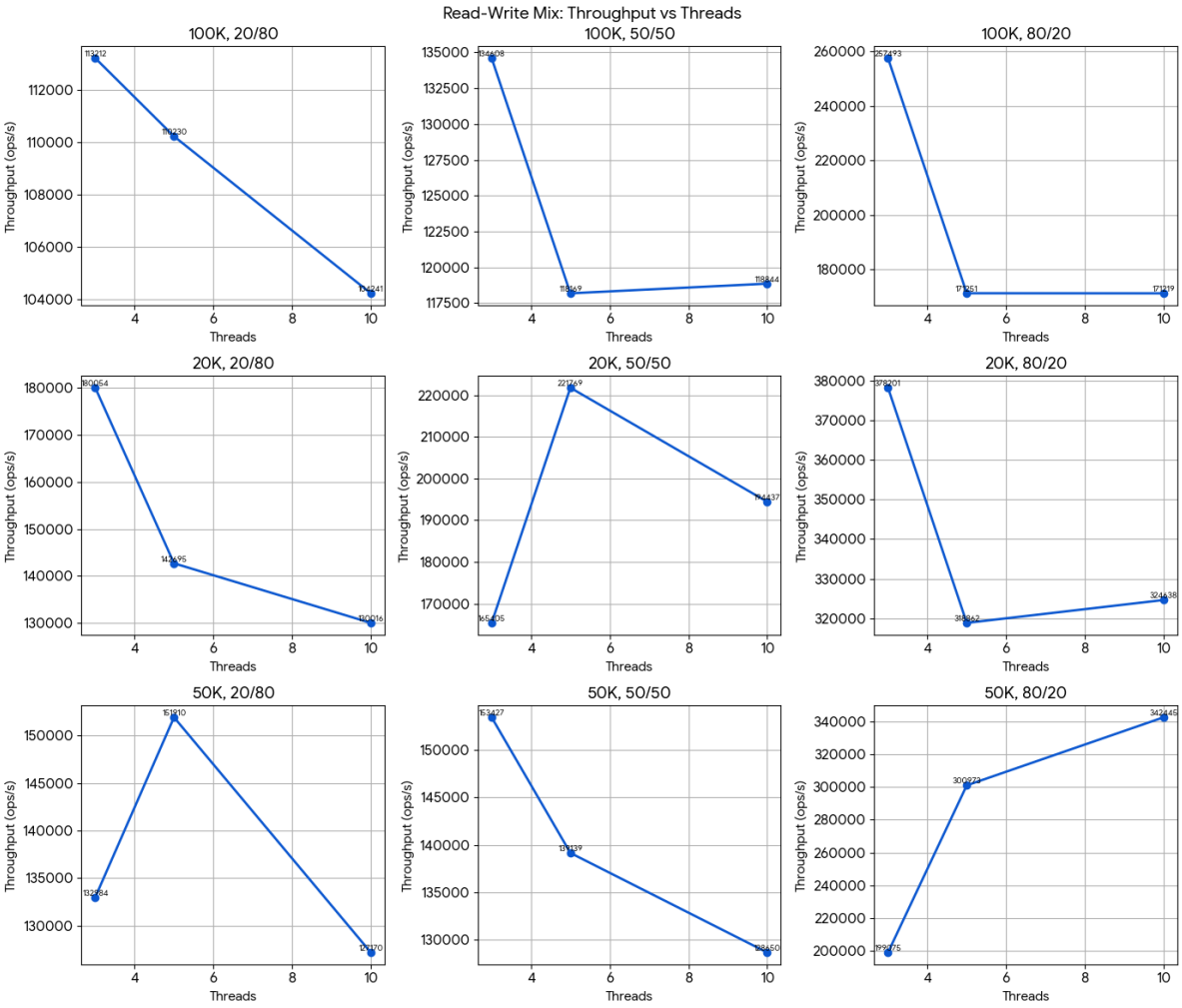
Reads: Παρατηρούμε ότι η απόδοση αυξάνεται σημαντικά καθώς αυξάνεται ο αριθμός των νημάτων μιας και οι αναγνώσεις γίνονται παράλληλα αφού απουσιάζουν οι εγγραφείς. Το read latency παραμένει σταθερό έχοντας μια αύξηση στα 10 threads καθώς αυξάνοντας αρκετά τον αριθμό τους, αρχίζουμε να μην έχουμε βελτιώσεις λόγω του context switching.

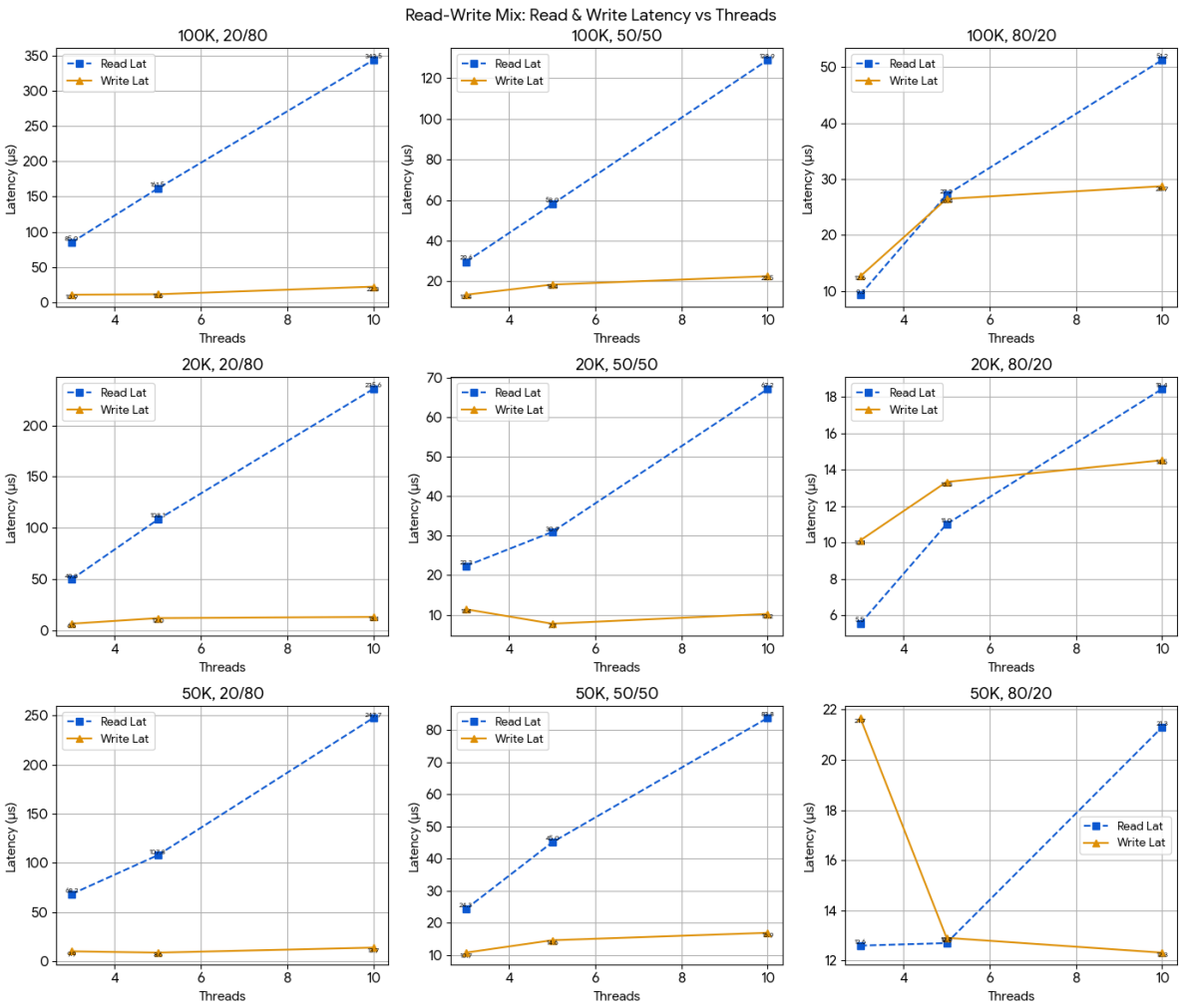
Writes: Για τις εγγραφές παρατηρούμε αυξημένους χρόνους καθώς επιτρέπουμε ένα μόνο εγγραφέα κάθε φορά χάνοντας έτσι την παραλληλία αλλά διασώζοντας την ακεραιότητα των δεδομένων. Αυτό συνεπάγεται και αύξηση στο latency με την αύξηση των νημάτων καθώς περιμένουν περισσότερη ώρα για να πάρουν lock. Παρατηρούμε επίσης αύξηση στο χρόνο με την αύξηση των εγγραφών καθώς περισσότερες εγγραφές οδηγούν σε compaction που είναι χρονοβόρο σε αντίθεση με μια απλή εγγραφή στο memtable.

Read-Write: Παρατηρούμε ότι καθώς ανεβαίνει το ποσοστό εγγραφών έναντι των αναγνώσεων αυξάνονται τα latency και πέφτει το throughput. Αυτό συμβαίνει για δύο λόγους. Αρχικά γίνονται περισσότερα compactions επομένως περιμένουν και οι γραφείς και οι αναγνώστες μέχρι να τελειώσει ο ενεργός γραφέας την παραπάνω διαδικασία, και δεύτερον λόγω της προτεραιότητας στους εγγραφείς που έχουμε προσθέσει. Το δεύτερο φαίνεται από την αύξηση στο latency των αναγνωστών καθώς μπαίνουν πρώτα οι εγγραφείς και μετά οι αναγνώστες.

Παρακάτω φαίνονται και οι γραφικές παραστάσεις των παραπάνω μετρήσεων:




Read-Write Mix: Throughput vs Threads
100K, 20/80
100K, 50/50
100K, 80/20
20K, 20/80
20K, 50/50
20K, 80/20
50K, 20/80
50K, 50/50
50K, 80/20
Throughput (ops/s)
Threads



Συνολικά η υλοποίηση είναι λειτουργική καθώς πληροί τις προδιαγραφές συγχρονισμού για read heavy workloads. Για write-heavy και balanced workloads χάνουμε σε ταχύτητα αλλά αποφεύγουμε το starvation.

ScreenShots

```
myy601@myy601lab1:~/kiwi/kiwi-source$ make
cd engine && make all
make[1]: Entering directory '/home/myy601/kiwi/kiwi-source/engine'
    CC db.o
    CC memtable.o
    CC indexer.o
    CC sst.o
    CC sst_builder.o
    CC sst_loader.o
    CC sst_block_builder.o
    CC hash.o
    CC bloom_builder.o
    CC merger.o
    CC compaction.o
    CC skiplist.o
    CC buffer.o
    CC arena.o
    CC utils.o
    CC crc32.o
    CC file.o
    CC heap.o
    CC vector.o
    CC log.o
    CC lru.o
    AR libindexer.a
make[1]: Leaving directory '/home/myy601/kiwi/kiwi-source/engine'
cd bench && make all
make[1]: Entering directory '/home/myy601/kiwi/kiwi-source/bench'
gcc -g -ggdb -Wall -Wno-implicit-function-declaration -Wno-unused-but-set-variable bench.c kiwi.c -L ../engine -lindexer -lpthread -lsnappy -o kiwi-bench
```

```
============== READ/WRITE TEST ===============
Threads      : 3
R/W Ratio    : 80% Reads / 20% Writes
Total Ops    : 20000 (Found: 0)
Total Time   : 0.084 sec
Throughput   : 237846 ops/sec
Avg Latency  : 11.21 us/op
Read Latency : 9.34 us/op
Write Latency: 18.55 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 20000 3 80 1
```

```
============== READ/WRITE TEST ===============
Threads      : 5
R/W Ratio    : 80% Reads / 20% Writes
Total Ops    : 20000 (Found: 0)
Total Time   : 0.057 sec
Throughput   : 349125 ops/sec
Avg Latency  : 11.25 us/op
Read Latency : 10.97 us/op
Write Latency: 12.44 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 20000 5 80 1
```

```
============== READ/WRITE TEST ===============
Threads      : 10
R/W Ratio    : 80% Reads / 20% Writes
Total Ops    : 20000 (Found: 0)
Total Time   : 0.082 sec
Throughput   : 244054 ops/sec
Avg Latency  : 31.66 us/op
Read Latency : 34.76 us/op
Write Latency: 18.98 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 20000 10 80 1
```

```
============== READ/WRITE TEST ===============
Threads      : 3
R/W Ratio    : 80% Reads / 20% Writes
Total Ops    : 50000 (Found: 0)
Total Time   : 0.235 sec
Throughput   : 213085 ops/sec
Avg Latency  : 13.34 us/op
Read Latency : 11.67 us/op
Write Latency: 20.12 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 50000 3 80 1
```

```
============== READ/WRITE TEST ===============
Threads      : 5
R/W Ratio    : 80% Reads / 20% Writes
Total Ops    : 50000 (Found: 0)
Total Time   : 0.210 sec
Throughput   : 238550 ops/sec
Avg Latency  : 18.10 us/op
Read Latency : 17.92 us/op
Write Latency: 18.79 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 50000 5 80 1
```

```
============== READ/WRITE TEST ===============
Threads      : 10
R/W Ratio    : 80% Reads / 20% Writes
Total Ops    : 50000 (Found: 0)
Total Time   : 0.212 sec
Throughput   : 236298 ops/sec
Avg Latency  : 28.99 us/op
Read Latency : 31.13 us/op
Write Latency: 20.39 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 50000 10 80 1
```

```
============== READ/WRITE TEST ===============
Threads       : 3
R/W Ratio     : 80% Reads / 20% Writes
Total Ops     : 100000 (Found: 0)
Total Time    : 0.520 sec
Throughput    : 192260 ops/sec
Avg Latency   : 14.54 us/op
Read Latency  : 13.21 us/op
Write Latency: 19.90 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 100000 3 80 1
```

```
============== READ/WRITE TEST ===============
Threads       : 5
R/W Ratio     : 80% Reads / 20% Writes
Total Ops     : 100000 (Found: 0)
Total Time    : 0.502 sec
Throughput    : 199366 ops/sec
Avg Latency   : 21.17 us/op
Read Latency  : 21.38 us/op
Write Latency: 20.35 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 100000 5 80 1
```

```
============== READ/WRITE TEST ===============
Threads       : 10
R/W Ratio     : 80% Reads / 20% Writes
Total Ops     : 100000 (Found: 0)
Total Time    : 0.600 sec
Throughput    : 166532 ops/sec
Avg Latency   : 47.58 us/op
Read Latency  : 51.93 us/op
Write Latency: 30.33 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 100000 10 80 1
```

```
============== READ/WRITE TEST ===============
Threads       : 3
R/W Ratio     : 50% Reads / 50% Writes
Total Ops     : 20000 (Found: 0)
Total Time    : 0.109 sec
Throughput    : 182988 ops/sec
Avg Latency   : 15.12 us/op
Read Latency  : 20.15 us/op
Write Latency: 10.15 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 20000 3 50 1
```

```
============== READ/WRITE TEST ===============
Threads       : 5
R/W Ratio     : 50% Reads / 50% Writes
Total Ops     : 20000 (Found: 0)
Total Time    : 0.102 sec
Throughput    : 195427 ops/sec
Avg Latency   : 17.92 us/op
Read Latency  : 25.57 us/op
Write Latency: 10.22 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 20000 5 50 1
```

```
============== READ/WRITE TEST ===============
Threads       : 10
R/W Ratio     : 50% Reads / 50% Writes
Total Ops     : 20000 (Found: 0)
Total Time    : 0.097 sec
Throughput    : 205335 ops/sec
Avg Latency   : 31.23 us/op
Read Latency  : 52.86 us/op
Write Latency: 9.82 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 20000 10 50 1
```

```
============== READ/WRITE TEST ===============
Threads       : 3
R/W Ratio     : 50% Reads / 50% Writes
Total Ops     : 50000 (Found: 0)
Total Time    : 0.299 sec
Throughput    : 166984 ops/sec
Avg Latency   : 15.58 us/op
Read Latency  : 20.79 us/op
Write Latency: 10.41 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 50000 3 50 1
```

```
============== READ/WRITE TEST ===============
Threads       : 5
R/W Ratio     : 50% Reads / 50% Writes
Total Ops     : 50000 (Found: 0)
Total Time    : 0.354 sec
Throughput    : 141376 ops/sec
Avg Latency   : 28.53 us/op
Read Latency  : 43.40 us/op
Write Latency: 13.82 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 50000 5 50 1
```

```
============== READ/WRITE TEST ===============
Threads       : 10
R/W Ratio     : 50% Reads / 50% Writes
Total Ops     : 50000 (Found: 0)
Total Time    : 0.360 sec
Throughput    : 139069 ops/sec
Avg Latency   : 50.81 us/op
Read Latency  : 86.37 us/op
Write Latency: 15.72 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 50000 10 50 1
```

```
============== READ/WRITE TEST ===============
Threads       : 3
R/W Ratio     : 50% Reads / 50% Writes
Total Ops     : 100000 (Found: 0)
Total Time    : 0.744 sec
Throughput    : 134463 ops/sec
Avg Latency   : 20.52 us/op
Read Latency  : 27.07 us/op
Write Latency: 13.97 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 100000 3 50 1
```

```
============== READ/WRITE TEST ===============
Threads       : 5
R/W Ratio     : 50% Reads / 50% Writes
Total Ops     : 100000 (Found: 0)
Total Time    : 0.765 sec
Throughput    : 130638 ops/sec
Avg Latency   : 35.15 us/op
Read Latency  : 53.52 us/op
Write Latency: 16.66 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 100000 5 50 1
```

```
============== READ/WRITE TEST ===============
Threads       : 10
R/W Ratio     : 50% Reads / 50% Writes
Total Ops     : 100000 (Found: 0)
Total Time    : 0.862 sec
Throughput    : 115953 ops/sec
Avg Latency   : 73.37 us/op
Read Latency  : 122.69 us/op
Write Latency: 24.27 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 100000 10 50 1
```

```
============== READ/WRITE TEST ===============
Threads      : 3
R/W Ratio    : 20% Reads / 80% Writes
Total Ops    : 20000 (Found: 0)
Total Time   : 0.103 sec
Throughput   : 195065 ops/sec
Avg Latency  : 12.34 us/op
Read Latency : 33.64 us/op
Write Latency: 7.07 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 20000 3 20 1
```

```
============== READ/WRITE TEST ===============
Threads      : 5
R/W Ratio    : 20% Reads / 80% Writes
Total Ops    : 20000 (Found: 0)
Total Time   : 0.120 sec
Throughput   : 166027 ops/sec
Avg Latency  : 25.35 us/op
Read Latency : 93.44 us/op
Write Latency: 8.46 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 20000 5 20 1
```

```
============== READ/WRITE TEST ===============
Threads      : 10
R/W Ratio    : 20% Reads / 80% Writes
Total Ops    : 20000 (Found: 0)
Total Time   : 0.152 sec
Throughput   : 131550 ops/sec
Avg Latency  : 54.13 us/op
Read Latency : 215.19 us/op
Write Latency: 13.07 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 20000 10 20 1
```

```
============== READ/WRITE TEST ===============
Threads      : 3
R/W Ratio    : 20% Reads / 80% Writes
Total Ops    : 50000 (Found: 0)
Total Time   : 0.305 sec
Throughput   : 163864 ops/sec
Avg Latency  : 17.01 us/op
Read Latency : 52.39 us/op
Write Latency: 8.24 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 50000 3 20 1
```

```
============== READ/WRITE TEST ===============
Threads      : 5
R/W Ratio    : 20% Reads / 80% Writes
Total Ops    : 50000 (Found: 0)
Total Time   : 0.352 sec
Throughput   : 141980 ops/sec
Avg Latency  : 30.82 us/op
Read Latency : 108.08 us/op
Write Latency: 11.39 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 50000 5 20 1
```

```
============== READ/WRITE TEST ===============
Threads      : 10
R/W Ratio    : 20% Reads / 80% Writes
Total Ops    : 50000 (Found: 0)
Total Time   : 0.358 sec
Throughput   : 139766 ops/sec
Avg Latency  : 58.76 us/op
Read Latency : 236.22 us/op
Write Latency: 14.07 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 50000 10 20 1
```

```
============== READ/WRITE TEST ===============
Threads      : 3
R/W Ratio    : 20% Reads / 80% Writes
Total Ops    : 100000 (Found: 0)
Total Time   : 0.729 sec
Throughput   : 137106 ops/sec
Avg Latency  : 21.40 us/op
Read Latency : 65.34 us/op
Write Latency: 10.31 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 100000 3 20 1
```

```
============== READ/WRITE TEST ===============
Threads      : 5
R/W Ratio    : 20% Reads / 80% Writes
Total Ops    : 100000 (Found: 0)
Total Time   : 0.780 sec
Throughput   : 128194 ops/sec
Avg Latency  : 36.00 us/op
Read Latency : 129.13 us/op
Write Latency: 12.96 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 100000 5 20 1
```

```
============== READ/WRITE TEST ===============
Threads      : 10
R/W Ratio    : 20% Reads / 80% Writes
Total Ops    : 100000 (Found: 0)
Total Time   : 0.871 sec
Throughput   : 114854 ops/sec
Avg Latency  : 73.13 us/op
Read Latency : 297.78 us/op
Write Latency: 17.80 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench readwrite 100000 10 20 1
```

```
================= READ TEST ==================
Threads      : 3
Total Ops    : 20000 (100% Reads, Found: 0)
Total Time   : 0.008 sec
Throughput   : 2373887 ops/sec
Read Latency : 0.07 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench read 20000 3 null 1
```

```
================= READ TEST ==================
Threads      : 5
Total Ops    : 20000 (100% Reads, Found: 0)
Total Time   : 0.011 sec
Throughput   : 1846040 ops/sec
Read Latency : 0.12 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench read 20000 5 null 1
```

```
================= READ TEST ==================
Threads      : 10
Total Ops    : 20000 (100% Reads, Found: 0)
Total Time   : 0.009 sec
Throughput   : 2327747 ops/sec
Read Latency : 0.42 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench read 20000 10 null 1
```

```
================= READ TEST ==================
Threads      : 3
Total Ops    : 50000 (100% Reads, Found: 0)
Total Time   : 0.018 sec
Throughput   : 2840425 ops/sec
Read Latency : 0.07 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench read 50000 3 null 1
```

```
================= READ TEST ==================
Threads      : 5
Total Ops    : 50000 (100% Reads, Found: 0)
Total Time   : 0.025 sec
Throughput   : 1972776 ops/sec
Read Latency : 0.13 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench read 50000 5 null 1
```

```
================= READ TEST ==================
Threads      : 10
Total Ops    : 50000 (100% Reads, Found: 0)
Total Time   : 0.024 sec
Throughput   : 2097315 ops/sec
Read Latency : 0.68 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench read 50000 10 null 1
```

```
================= READ TEST ==================
Threads      : 3
Total Ops    : 100000 (100% Reads, Found: 0)
Total Time   : 0.032 sec
Throughput   : 3106941 ops/sec
Read Latency : 0.08 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench read 100000 3 null 1
```

```
================= READ TEST ==================
Threads      : 5
Total Ops    : 100000 (100% Reads, Found: 0)
Total Time   : 0.046 sec
Throughput   : 2154290 ops/sec
Read Latency : 0.09 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench read 100000 5 null 1
```

```
================= READ TEST ==================
Threads      : 10
Total Ops    : 100000 (100% Reads, Found: 0)
Total Time   : 0.042 sec
Throughput   : 2371691 ops/sec
Read Latency : 0.41 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench read 100000 10 null 1
```

```
================= WRITE TEST =================
Threads      : 3
Total Ops    : 20000 (100% Writes)
Total Time   : 0.099 sec
Throughput   : 201493 ops/sec
Write Latency: 12.92 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench write 20000 3 null 1
```

```
================= WRITE TEST =================
Threads      : 5
Total Ops    : 20000 (100% Writes)
Total Time   : 0.125 sec
Throughput   : 159612 ops/sec
Write Latency: 26.16 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench write 20000 5 null 1
```

```
================= WRITE TEST =================
Threads      : 10
Total Ops    : 20000 (100% Writes)
Total Time   : 0.125 sec
Throughput   : 160555 ops/sec
Write Latency: 46.82 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench write 20000 10 null 1
```

```
================= WRITE TEST =================
Threads      : 3
Total Ops    : 50000 (100% Writes)
Total Time   : 0.278 sec
Throughput   : 179634 ops/sec
Write Latency: 14.96 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench write 50000 3 null 1
```

```
================= WRITE TEST =================
Threads      : 5
Total Ops    : 50000 (100% Writes)
Total Time   : 0.281 sec
Throughput   : 178090 ops/sec
Write Latency: 21.52 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench write 50000 5 null 1
```

```
================= WRITE TEST =================
Threads      : 10
Total Ops    : 50000 (100% Writes)
Total Time   : 0.355 sec
Throughput   : 141017 ops/sec
Write Latency: 56.29 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench write 50000 10 null 1
```

```
================= WRITE TEST =================
Threads      : 3
Total Ops    : 100000 (100% Writes)
Total Time   : 0.563 sec
Throughput   : 177757 ops/sec
Write Latency: 15.54 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench write 100000 3 null 1
```

```
================= WRITE TEST =================
Threads      : 5
Total Ops    : 100000 (100% Writes)
Total Time   : 0.581 sec
Throughput   : 172078 ops/sec
Write Latency: 27.57 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench write 100000 5 null 1
```

```
================= WRITE TEST =================
Threads      : 10
Total Ops    : 100000 (100% Writes)
Total Time   : 0.692 sec
Throughput   : 144446 ops/sec
Write Latency: 58.49 us/op
==============================================
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ make clean && make && ./kiwi-bench write 100000 10 null 1
```